UChicago Knowledge Lab Computing Manual

Note: this is a living document. Please add to it and update it as you see fit.

Contents:

Compute resources

Midway3

- The university's main research computing cluster
- Midway3 connection information and official documentation can be found here: https://rcc-uchicago.github.io/user-guide/ssh/main
- Midway uses the SLURM job scheduler
    - There is excellent documentation for SLURM on google. Almost anything you want to do is possible, it just takes a bit of searching.
    - The main thing is to request the right amount of resources, because the cluster is a shared resource.
        - Obviously you should request enough resources to do your job.
        - If I have code that uses one CPU, I should request 1 or 2 CPUs, because that is all I need, and then the rest of the CPUs on a machine can be used by someone else.
        - For GPUs also, if your code is not written to use multiple GPUs, it is best to request a single GPU, so that other people can use other GPUs on the same node. If you don't need any GPUs, it is best not to use a GPU node.
        - Sometimes you want to use all the resources on a single node, in which case you can use the `--exclusive` command in slurm to reserve a whole node.

Nodes

A node is a physical server/computer. Different nodes have different capabilities, and it's important to pick the right one for your job.

Types of node:

- There are two "**login**" nodes – these are the ones you first connect to when you connect to midway3. They're the only nodes with internet access, so use these for downloading data and so on. However, they are not supposed to be used for computing (that can slow down everyone else's access to the whole cluster). To run your code, submit an interactive or batch job to nodes on one of the partitions.
- "**caslake**" partition nodes have 48 cores and 192 GB of RAM/memory.
- "**bigmem**" nodes have 48 cores and 1.5TB or 2TB of RAM
- "**amd**" nodes have 128 cores and 256 GB RAM. They use amd processor architecture instead of intel, which means each core runs slower, but you get more cores. This is better for some jobs, worse for others.
- "**gpu**" nodes have 4 GPUs each, but different nodes have different kinds of GPUs, different numbers of CPUs, and different amounts of RAM.
    - There are nodes with rtx6000, v100, a100, and h100 GPUs. Of course the h100's are biggest and fastest, but there are less of those nodes than other kinds, and

we don't always need an a100 or h100. I get similar tokens per second doing LLM inference with a 70B model on 1 H100, 2 A100s, or 3 rtx6000s.
    - You can request a particular kind of GPU by using a constraint flag such as `--constraint=v100` in your slurm submission.

Partitions

About partitions:

- The slurm cluster is divided into partitions. Each partition has a different kind of node, and a different set of users. It is important to pick the right partition for your work, so that you get the resources you need, and share best with other users.
- Sometimes the nodes in one partition are busy, but the ones in another are available, so it is worth trying alternatives if your first try is taking too long.
    - You can see the use on different partitions with some helpful slurm commands.
        - `squeue | grep [partition name]` will show you all the jobs currently running on a particular partition. You can also grep for your own username if you want to see your own jobs, etc.
- Most partitions have a walltime limit of 36 hours, meaning you can submit jobs that request up to 36 hours. For jobs that need more time than that, it is best to design them to do work in chunks or checkpoints, so that they can stop and then pick up again where they left off.
- The main documentation of available nodes and partitions is here: https://rcc-uchicago.github.io/user-guide/partitions

Interactive and batch jobs:

- There are two ways to run code on midway (i.e. ask for resources from a partition), batch jobs, and interactive jobs.
- **Batch jobs**:
    - Documentation here
    - This is great if you have some code you want the system to run on its own. Set it and forget it style. You tell the system what code to run, what resources (memory, cpu, gpu, time, etc.) the code needs, and the system will run it as soon as possible. Your code should save its results to a file so you can see it later. Slurm will also save the standard output and any errors from your code to files for you.
    - Batch jobs end either when your code finishes or when the time limit you set run out. This is efficient: it frees up cluster resources for other users when you are done using them.
    - **Job arrays**: are a special kind of batch job. If you want to run the same code multiple times (e.g. with different input data, different random seeds, whatever), this is for you.
        - Documentation here
        - Limits

        - The caslake and ssd partitions have per-user limits to make sure other users code can still run while your array is running. Ssd will give you up to 5 whole nodes worth of resources (e.g. 240 cores) at a time. Caslake will give you a lot more, especially on nights, weekends, and holidays. For example, one Friday it had no nodes free during the day, but it 54 nodes free at night.
        - The jevans partition has no limits per user, and only 4 nodes. If you send a big job array to this partition, it will use the whole partition and leave no space for other users' jobs. To share the partition with other users, you can set an array limit, e.g. "--array=0-999%4" will queue up 1,000 jobs to run, but only allow 4 to run at a time.
- **Interactive jobs**:
    - Documentation here
    - This is good if you need to interact with your code in real time. For example, if you want to open jupyter, write a little code, run it, change the code, run again, etc.
    - Interactive jobs end when you disconnect, or when you end them. Sometimes people use screen or tmux to keep interactive sessions alive when they disconnect. This is okay, but please remember to end your interactive job when you are done running code. You can always start another one later. (sometimes people forget, then leave their interactive jobs running for days when they are not using them, wasting resources and preventing other users from working)
- **Resources**:
    - Whenever you submit a job to slurm, you need to tell it what resources the job needs. How many CPUs and GPUs? How much memory and time?
    - With memory and time, a good general rule is to ask for about 20% more than you think you need. You don't want the job to run out of memory or time before your code finishes, but you also don't want to request more resources than you need, because that prevents other people from using them.
    - With CPU, a good rule is to ask for what you think you need, plus one more.
        - If you have single threaded (not parallel) code, 2 CPUs is a good number. This is because your code will use one, and the other will be available to handle system overhead and the random side things going on.
    - With GPU, it is usually best to ask for only one, unless you know your code actually needs or uses more than one.
        - A lot of torch code for example does not use a full GPU and only uses one GPU at a time. This means you could run multiple GPU programs at the same time on the same GPU if you wanted to.
        - Some LLM code can use one or multiple GPUs, but when it uses two, each only gets half used and the end result is no faster than using one.
        - The command nvitop will let you see how much of the GPU your code is using in real time.

- If you need all of the memory or all of the cores on a node, then you should use the --exclusive slurm flag to request the entire node to yourself. Only do this if your code will actually use the full memory or full CPUs of a node.

List of partitions:

- University-wide partitions are shared across the whole campus. There are many nodes, but also many users, so they can be busy at times. This is our main computing allocation, and you can use these partitions with the account “pi-jevans”.
  - **caslake**: regular compute nodes. 36 hour walltime limit
  - **gpu**: gpu nodes. In this partition there are different kinds of gpus. It will give you one of these at random unless you specifically ask for a particular card with --constraint=[cardname]
    - 5 nodes with rtx6000,
    - 5 nodes with v100,
    - 1 node with an a100.
  - **bigmem**: bigmem nodes. 36 hour walltime limit
  - **amd**: amd nodes. 36 hour walltime limit
- The ssd partitions are shared just in the social sciences division. It has very few users, so it is often a good choice. To use these, set the account to “ssd” not “pi-jevans”.
  - **ssd** has 18 regular compute nodes. 36 hour walltime limit
  - **ssd-gpu** has a single gpu node with 4 A100 cards. 36 hour walltime limit. Note that you need to set --qos=ssd, not the default --qos=ssd-gpu
- The lab has its own partitions
  - **jevans** is just for people in the lab. It has 4 bigmem nodes (2 with 1.5TB RAM, 2 with 2TB). 48 cores each, 48 hour walltime limit.
  - **jevans-gpu** is a gpu partition with one node. It has 4 H100 GPUs with 94GB of VRAM each, 500GB of RAM, and 32 CPU cores. 48 hour walltime limit. Try not to use this partition for jobs that do not require a GPU.
- Available partitions
  - Here is a useful script for checking the availability on different partitions. Change it as you like. It reports the number of idle nodes on each partition. Note that this only shows nodes that are completely free. Many nodes will be ‘mixed’, meaning they are partly used, partly free.
  - $ cat check_idle.sh

```
#!/bin/bash
echo "caslake"
sinfo -p caslake -N | grep idle | wc -l
echo "ssd"
sinfo -p ssd -N | grep idle | wc -l
echo "jevans"
sinfo -p jevans -N | grep idle | wc -l
echo "ssd gpu"
sinfo -p ssd-gpu -N | grep idle | wc -l
echo "caslake gpu"
```

```
sinfo -p  gpu -N | grep idle | wc -l
echo "jevans gpu"
sinfo -p jevans-gpu -N | grep idle | wc -l
```

Storage space

- You can get a readout of all available and used storage for you and for the entire lab with withthe “quota” command on the terminal.
- Lab storage on midway is in the directory /project/jevans/ . Presently we have 100TB of storage space to share with the whole lab. We can purchase more as needed, but try to be kind and conserve space when possible.
- Tips for conserving space:
    - Compress data files
    - Delete large files that you no longer need
- Midway also has 2TB of scratch storage space for every user, where you can temporarily hold large amounts of data. Your scratch space is located at /scratch/midway3/username/ (e.g. Jeff’s is /scratch/midway3/jlockhart/). Unline your home and the project directory, scratch is not backed up. Things there could disappear or be lost. I’ve never seen it happen, but it’s still worth remembering to keep your important things elsewhere.

Compute Allocation

We get yearly allocations of compute from RCC. This does not include the usage of the jevans partition but does include things like caslake, bigmem, gpu, etc.

- We have to apply to renew our allocation every year in September.
- `accounts allocations` tells us our current allocation of compute
- `accounts balance` tells us how much of our compute allocation we have used so far.
- More details here: https://rcc.uchicago.edu/accounts-allocations/request-allocation

Knowledge Garden

- A bare metal server physically sitting in the knowledge lab
    - Shared system, so please watch the resource availability while you work with tools like “htop” and “nvitop,” and close your programs when they’re finished.
- Has 1 NVidia 3090 GPU
- Has 48 CPU cores
- Has 126GB RAM
- To connect:
    - First, get an account from someone who has sudo powers (e.g. Austin, Jamshid, Donghyun, several others)
    - Then, from the university network or VPN, ssh to 205.208.1.121

K-Garden 1080

- A bare metal server physically sitting in the knowledge lab
    - Shared system, so please watch the resource availability while you work with tools like “htop” and “nvitop,” and close your programs when they’re finished.
- Has 4 NVidia 1080 Ti GPUs
- Has 12 CPU cores
- Has 126GB RAM
- To connect:
    - First, get an account from someone who has sudo powers (e.g. Austin, Jeff, Donghyun, Junsol)
    - Then, from the university network or VPN, ssh to 205.208.1.203

SSCS Acropolis

- A cluster specifically for social science.
- Does not cross-mount drives with midway, so your files on one are not visible on the other.
- PBS (not slurm) job scheduler
- Documentation: https://sscs.uchicago.edu/category/faq/cluster

Data sets

- **Microsoft Academic Graph Dec** 2021 version in Midway3: `/project/jevans/MAG_Dec_2021_snapshot/`
    - Data scheme is available: *"Microsoft Academic Graph data schema - Microsoft Academic Services _ Microsoft Docs.html"*

- **OpenAlex**:
    - There is a copy from October 2023 in /project/jevans/tip/data/openalex/. The raw data is there, as well as some cleaned up parquet files. You can read those files, but please do not change or delete them as other people are using them too.
    - (I think) the raw data is in /project/jevans/tip/data/openalex/data

- **Web of Science:**
    - A full dump (the XML files and a better version converted to parquet format) available on midway at /project/jevans/tip/data/wos_2023/
    - a version available within the SSD Cronus system (point of contact was Lucas Coady but not sure if he is still active; it looks like the bash script for generating the permission key does not work anymore).

- **IRIS Umetrics:** the lab has access to IRIS Umetrics data (https://iris.isr.umich.edu/); need to contact the UMichigan side to add names on the DUA if someone wants to use this.

- **Reddit** Dump on Midway3 (From onset to 2023-02; collected by Hongkai Mao): `/project/jevans/hongkai/reddit`.
- **Reddit** Dump on Midway3 (From 2024-04 to 2025-04; collected by Ruining He (rnhe)): `/project/jevans/reddit_data`. Future maintenance advice: use aria2c to download the Reddit dump from https://academictorrents.com/ on multiple AWS EC2 ubuntu instances, then transfer them to midway3 using scp command.

- **Relevant UChicago-accessible datasets:** ProQuest TDM (newspapers, press releases), Refinitiv (earnings call transcripts),

- **PubMed Knowledge Graph Version 2 (up to 2023)**: `/project/jevans/PKG_v2_2023` this data connects biomedical publications, patents, and clinical trials.

- **Peer Review data (comment text, score, and confidence) for computer science conferences**: /project/jevans/Honglin_Bao_share/peer_review

Midway software and application tips

Helpful Slurm Commands

- myq
    - Shows all of your current running and pending jobs
- squeue | grep jevans
    - Shows all current and pending jobs on the jevans partitions. This lets you see what other people in the lab are doing and how busy things are.
    - Can also run this command with “ssd” or “caslake” instead of “jevans” to see those partitions.
- sinfo | grep jevans
    - Shows the status of all nodes on the jevans partitions. This lets you see which nodes are idle, busy, or partly-busy (“mixed”)
    - Can also run this command with “ssd” or “caslake” instead of “jevans” to see those partitions.
- scontrol show job [jobid]
    - Lets you see detailed information about any job, past or future.
-

Installing things

- Many new users are tempted to simply install the software and packages they need for their work. This seems reasonable when you are new. However, it is a bad idea on midway. Your home directory is small, and installing software will quickly fill it up so that you exceed your disk quota.
    - Hint: use the “quota” command to see your space and file count limits on midway.
- Most software and packages you want to use are already installed on the system. All you need to do is load them onto your path, using the “module” command. E.g. “module load spark” will load spark, “module load python/anaconda-2023.09” will load that specific version of python, and so on.
- Search the RCC website or use “module avail” to see what modules are already installed.
- If you use python, you may need a few packages that are not included in the modules. If you load the module first, then install the package you need, typically pip or conda will only install the extra packages you need, thus minimizing the total space you use.

Jupyter lab

- Note: there may be a canonical way to run jupyter on midway now, but there wasn’t when I asked in 2022, so here’s the solution that I hacked together that works for me.
- First, start an interactive session in slurm. Once it starts, run this script:

- $ cat start-jup.sh
  #!/bin/bash

  #port must be between 15000 and 30000. Others blocked by firewall
  port=18765
  ip=$(/sbin/ip route get 8.8.8.8 | awk '{print $7;exit}')

  #load modules you want here
  module load python/anaconda-2022.05

  #start jupyter lab
  jupyter lab --no-browser --port $port --ip $ip
- This will start a jupyter lab server and give you a link to it that you can click and access in your laptop/desktop web browser.
- Note that interactive slurm jobs die when you disconnect from them. If you have an unstable connection, or you plan to move around with your laptop and you want your code to keep running, you should use a tool like screen or tmux to keep the job alive. You can then reconnect to the jupyter lab session via your browser later.

(py)spark

- It is possible to use spark on slurm! Yay! Below are some tips for setting up spark with an interactive jupyterlab session.
  - Start a job with exclusive access to one or more nodes. (I find one node is often enough, but some things definitely benefit from more.)
    - sinteractive --partition=caslake --account=pi-jevans --exclusive --nodes=5 --time=8:00:00
    - I find that the ssd and caslake partitions are often best for multi-node requests, because they're biggest and most likely to have free nodes
    - The AMD partitions work, and they have many more processors per node, but depending on what you're doing, the overall throughput per node can be lower than the other partitions.
    - I have not tested, but spark can use GPUs now, so if you're doing a matrix algebra heavy workload (e.g. with sparkML), that might be worth trying.
  - Once your job starts, run this script on the head node to start a spark cluster and jupyterlab server that can access it.
    - $ cat pyspark_startup.sh

```
#!/bin/bash

# must be between 15000 and 30000
port=18765
ip=$(/sbin/ip route get 8.8.8.8 | awk '{print
$7;exit}')
```

```
# load modules
module load python/anaconda-2022.05
module load spark

# This command starts the spark workers on the
allocated nodes
start-spark-slurm.sh
# This syntax tells the spark workers where the
master is
export MASTER=spark://$HOSTNAME:7077
export SPARK_WORKER_DIR=$SLURM_SUBMIT_DIR/work

#spark setup
export PYSPARK_DRIVER_PYTHON=jupyter
export PYSPARK_DRIVER_PYTHON_OPTS="lab --no-browser
--port ${port} --ip ${ip}"

pyspark --packages
com.databricks:spark-xml_2.12:0.16.0
```

- Jupyter will launch and give you a link you can click to access it on your local browser.
- Common issues
  - "Has not accepted any resources" error:
    - try shutting down all the other kernels in your jupyterlab session. One of them is probably hogging the executors.
    - Also possible you set something in the spark session that is impossible (e.g. you asked for executors with more memory than exists on the nodes)
- In your python code, I recommend setting your executor memory to 100gb. The setup will be one executor per node, and I've found 100GB is as high as I can go without it complaining on the regular caslake nodes. More memory for the executors means fewer error messages and less data spilling to disk / slowing down your work.
  - spark = SparkSession.builder.config("spark.executor.memory", "100g").appName("pyspark").getOrCreate()

MySQL Database Support

(Updated by Donghyun Kang Nov. 29, 2023)

A module for mysql version 5.7 is installed in Midway3. Users can access these executables after loading it on the Midway login nodes or computing nodes in interactive sessions. Please read the following instructions!

- Configurations
    - When using MySQL, users run their own server application to host the database server. Users need to run it on a computing node (recommended!) in an interactive session. Once users log out, this process will be terminated. Then the database cannot be accessed until the user logs in again and restarts the server.

    - Please log in to Midway3 (interactive session) and run **"module load mysql"** before you move on to the next steps.

- Step 1: Prepare a folder for disk files
    - You need to prepare a folder that will be used to contain your database hard-disk files. The folder can be anywhere in our file system, **Home**, **Scratch** or **Project**.

    - It is recommended to use your project space because it supports snapshots and tape-backup. For example, you can use or create a folder at:
      /project/<PI-account>/<user_name>/mysql_data/ (ours will be /project/jevans/..)

- Step 2: Edit the configuration file
    - **Open the MySQL configuration file in your *Home directory: ~/.my.cnf .**
      If it doesn't exist, please create a new one. The file should include the following:

```
[mysqld]
socket=/home/<CNetID>/.sql.sock
/project/<PI-account>/<user_name>/mysql_data/
max_connections=4

[client]
socket=/home/<CNetID>/.sql.sock
```

    - It contains two segments. The first one with **mysqld** is to set up the server application (mysqld); the second one **client** is to set up the client application (mysql). In the segment of [mysqld], you need to specify a socket file. This file doesn't need to exist, but be sure the path exists and is writable by you. Another entry is the data directory that you prepared for disk files in Step 1.

    - In the second segment, you just need to specify the socket file, which must be the same as the one you set up in the first segment.

- Step 3: Initiate your database
    - (1) If you already have a complete set of data files (for example, you're migrating a MySQL database hosted by other servers), just copy them into the data directory you prepared in Step 1.

    - (2) If you start from an empty database, please execute the following command to initiate the disk files. (The --datadir is the same path you prepared in Step 1)

```
/software/mysql-5.7-el8-x86_64/bin/mysqld --initialize \
    --basedir=/software/mysql-5.7-el8-x86_64 \
    --datadir=<your_data_folder_path>
```

        - At the end of the output, you will see a temporary password for the root user of the database. Save it!

- **Step 4: Connect to a compute node and start server application**
    - Run “sinteractive” for a computing node.

    - Run "mysqld --defaults-file=~/.my.cnf &" in your terminal after loading the mysql module (“module load mysql”). This will launch the database server. Make sure to include the ampersand (“&”). If omitted, you will need to start another terminal and ssh to the Midway compute node (e.g., ssh midway3-0123) where you’re running the myql server.

- **Step 5: Connect to the server**
    - In a terminal logged into the node on Midway where you’re running the mysql server, do “module load mysql" (if you are using the same terminal from Step 4, no need to do this again) then **“mysql --defaults-file=~/.my.cnf”**

    - Or if you have a new empty database as previously described:
        - **“mysql --defaults-file=~/.my.cnf -u root -p”**. The password is the temporary one that you saved when the new database for yourself was created.

    - Again, users can only connect the server on the same node that they run the server application (mysqld). For example, do “echo $HOSTNAME” first before running the server application. If it is "midway3-####", the user will need to login to the same one with “ssh midway3-####” where #### is the number of the computing node where mysqld is on.

- **Step 6: Shut down the server**

- "pkill mysqld" in the command line to shut down the server application and log out.

- **Restrictions**
  - Users are not allowed to set up MySQL using a network port instead of the system socket file. The reason is that it exposes the user's database to the public. Don't use login nodes for any data or compute intensive work. System administrators may kill processes on a login node without warning if they use too many resources. Use a computing node with an interactive session for all local mysql servers.

Python Connector for MySQL Database

Users can connect to a MySQL DB using Python.

- Step 1: Start the DB
  - Once getting into a computing node, do the following to run the SQL server. Please read the above documentation if you have not established your socket configuration.

```
module load mysql

mysqld --defaults-file=~/.my.cnf &
```

- Step 2: Open Jupyter Notebook/Lab
  - Please refer to the other section of this document.

- Step 3: import sql.connector
  - There are some available Python libraries but here we use one of them.

```
import mysql.connector
Import pandas as pd

my_db = mysql.connector.connect(
  host="local",
  user="#username", #CNET ID
  password="#password", #Your password for the database (Not CNET)
  database="#database_name"
)

my_cursor = mydb.cursor()

my_cursor.execute("SELECT * FROM #table_name")
result = my_cursor.fetchall()
```

- Alternatively,

```
import mysql.connector
Import pandas as pd

#using pd.read_query
my_db = mysql.connector.connect(
  host="local",
  user="#username", #CNET ID
  password="#password", #Your password for the database (Not CNET)
  database="#database_name"
)

my_cursor = mydb.cursor()

query = "SELECT * FROM #table_name"
df  = pd.read_sql_query(query, my_db)
```